ASSIGNMENT 2 – CO2

TASK 01 – BUILDING TEXT UNDER LOAD: STRING, STRINGBUILDER OR STRINGBUFFER?

Course: CSA0903 – Programming in Java
Problem Context: Railway e-Ticket Generation

Name: Sowmiya. D

Register Number: 192511063

1. Aim

To compare String concatenation, StringBuilder and StringBuffer while generating railway e-tickets, measure their execution time under different workloads, and recommend suitable classes for individual ticket construction and a shared multithreaded log.

2. Problem Description

A railway reservation system combines details such as PNR, passenger name, train number, journey date, coach, berth and fare into a printable ticket. Repeatedly joining text with the + operator inside a loop may create many temporary String objects. This experiment compares that approach with the mutable StringBuilder and StringBuffer classes.

3. Prediction Before Implementation

String is immutable. Whenever concatenation creates a new result, the previous String cannot be modified. Therefore, repeated concatenation in a loop can create temporary objects and repeatedly copy previously accumulated characters. The cost is expected to increase noticeably as the number of fields grows.

StringBuilder is mutable. Its append() method adds data to an internal buffer, reducing the need to create a new String for every concatenation. It is expected to perform well for building a ticket within one thread.

StringBuffer is also mutable, but its methods are synchronized. It provides thread-safe individual operations and may be slightly slower than StringBuilder because of synchronization overhead.

Prediction: for a very short ticket, the difference may be small. For many tickets or many fields, StringBuilder and StringBuffer are expected to outperform repeated String concatenation.

4. Experimental Workloads

Workload	Tickets	Fields per ticket	Purpose
1 – Short ticket	1	6	Represents a small ticket; differences may be negligible.
2 – Heavy generation	10,000	50	Tests repeated ticket generation under load.
3 – Many-field ticket	2,000	100	Tests the effect of increasing fields per

			ticket.

5. Implementation

The Java program is saved separately as Main.java. It implements three methods: buildWithString(), buildWithStringBuilder() and buildWithStringBuffer(). Each method constructs a ticket using the requested number of fields.

The program performs two warm-up runs and then five measured runs for each approach. It calculates the average elapsed time using System.nanoTime(). The same workload is used for all three approaches to make the comparison fair.

6. How to Compile and Execute

1. Open the Task01 folder in Visual Studio Code.

2. Open the terminal in VS Code.

3. Compile the program: javac Main.java

4. Execute the program: java Main

5. Record the output and capture a screenshot of the terminal.

7. Experimental Results

Enter the average execution times printed by your own computer after running Main.java. Do not copy another person's timings because results depend on the processor, JVM, memory and background applications.

Workload	String (ms)	StringBuilder (ms)	StringBuffer (ms)
Workload 1 – 1 ticket, 6 fields	Fill after execution	Fill after execution	Fill after execution
Workload 2 – 10,000 tickets, 50 fields	Fill after execution	Fill after execution	Fill after execution
Workload 3 – 2,000 tickets, 100 fields	Fill after execution	Fill after execution	Fill after execution

8. Intermediate Objects and Memory

With repeated + concatenation in a loop, each iteration produces a new concatenated String result. Approximately one loop-result String is created per field, in addition to temporary objects potentially created while forming each field expression. Exact allocation counts depend on compiler and JVM optimizations.

StringBuilder and StringBuffer maintain a mutable character buffer and return a final String from toString(). Their buffers may resize as content grows. StringBuffer synchronizes its methods; StringBuilder does not.

The program prints an approximate JVM used-memory observation. This is not an exact per-object memory measurement: garbage collection, heap expansion and JVM optimizations can affect the result. Execution time averaged over repeated runs is the primary comparison.

9. Comparison of Prediction and Observation

After execution, compare the measured values with the prediction. If the short workload shows only a small difference but the larger workloads show a larger gap, the observation supports the prediction that repeated immutable concatenation becomes less efficient as the amount of text increases. If results vary, repeat the experiment and explain possible JVM warm-up, garbage collection and system-load effects.

10. Design Decision for a Multithreaded Booking Server

For constructing each individual e-ticket, StringBuilder is recommended. Each ticket can be built in a local StringBuilder owned by the thread generating that ticket. Sharing the builder is unnecessary, so synchronization is not required.

For a shared booking log that multiple threads append to, a single shared StringBuffer offers synchronized append operations. In a production server, a thread-safe logging framework or a queue-based logging design is generally preferable because it can manage concurrent writes, file output and log rotation more reliably. Synchronizing only individual append calls does not automatically make a multi-step logging operation atomic.

11. Alternative Concepts

String.join() can combine a collection or array of strings using a delimiter. StringJoiner is useful when building a delimited sequence and supports prefix and suffix. String.format() is useful for formatted output, but formatting repeatedly inside a large loop may add overhead. For this ticket-generation workload, StringBuilder is a direct and efficient choice.

12. Time and Space Complexity

Approach	Time complexity	Extra space	Reason
Repeated String +	$O(n^2)$ character copying (equal-sized fields)	$O(n^2)$ cumulative temporary allocation in a simple model; live space depends on GC	Previously built text is copied repeatedly.
StringBuilder	O(n) amortized for n appended characters	O(n) buffer plus final String	Mutable buffer avoids copying the full prefix each time.
StringBuffer	O(n) amortized for n appended characters, plus synchronization overhead	O(n) buffer plus final String	Mutable synchronized buffer.

Here n denotes the total number of characters appended, assuming append operations are amortized constant time per character. If n denotes the number of equal-length fields, repeated concatenation copies prefixes of increasing length, giving quadratic total copying.

13. Conclusion

The experiment compares immutable String concatenation with the mutable StringBuilder and StringBuffer classes under three workloads. The final recommendation is to use a local StringBuilder for each ticket and a thread-safe logging mechanism for shared log output. The conclusion must be checked against the measured results recorded above.

14. Coding

```
import java.util.*;


public class Main {


   static volatile long result;


   static String buildString(int fields) {
      String ticket = "";


      for (int i = 1; i <= fields; i++) {
         ticket = ticket + "Field" + i + " ";
      }


      return ticket;
   }


   static String buildStringBuilder(int fields) {
      StringBuilder ticket = new StringBuilder();


      for (int i = 1; i <= fields; i++) {
         ticket.append("Field").append(i).append(" ");
      }


      return ticket.toString();
   }
```

```
static String buildStringBuffer(int fields) {
    StringBuffer ticket = new StringBuffer();

    for (int i = 1; i <= fields; i++) {
        ticket.append("Field").append(i).append(" ");
    }

    return ticket.toString();
}

static long measure(int method, int tickets, int fields) {
    long start = System.nanoTime();
    long total = 0;

    for (int i = 0; i < tickets; i++) {
        String ticket;

        if (method == 1)
            ticket = buildString(fields);
        else if (method == 2)
            ticket = buildStringBuilder(fields);
        else
            ticket = buildStringBuffer(fields);

        total += ticket.length();
    }

    result = total;
```

```
    return System.nanoTime() - start;
}

static void test(int tickets, int fields) {

    System.out.println("\nTickets: " + tickets +
            " | Fields: " + fields);

    // Warm-up
    for (int i = 0; i < 3; i++) {
        measure(1, tickets, fields);
        measure(2, tickets, fields);
        measure(3, tickets, fields);
    }

    String[] names = {
        "String",
        "StringBuilder",
        "StringBuffer"
    };

    for (int method = 1; method <= 3; method++) {

        long totalTime = 0;

        for (int run = 0; run < 5; run++) {
            totalTime += measure(method, tickets, fields);
        }
```

```
            double average =
                    totalTime / 5.0 / 1_000_000.0;

            System.out.printf("%-15s : %.3f ms%n",
                    names[method - 1], average);
        }
    }

    public static void main(String[] args) {

        System.out.println("RAILWAY E-TICKET PERFORMANCE TEST");
        System.out.println("---------------------------------");

        test(1, 5);
        test(10000, 50);
        test(20000, 100);

        System.out.println("\nPerformance testing completed.");
    }
}
```

Output:

```
RAILWAY E-TICKET PERFORMANCE TEST
---------------------------------
Tickets: 1 | Fields: 5
String          : 0.047 ms
StringBuilder   : 0.009 ms
StringBuffer    : 0.019 ms
```

Tickets: 10000 | Fields: 50

String : 28.692 ms

StringBuilder : 12.394 ms

StringBuffer : 14.180 ms

Tickets: 20000 | Fields: 100

String : 133.052 ms

StringBuilder : 52.389 ms

StringBuffer : 105.849 ms

Performance testing completed.

Implementation:

